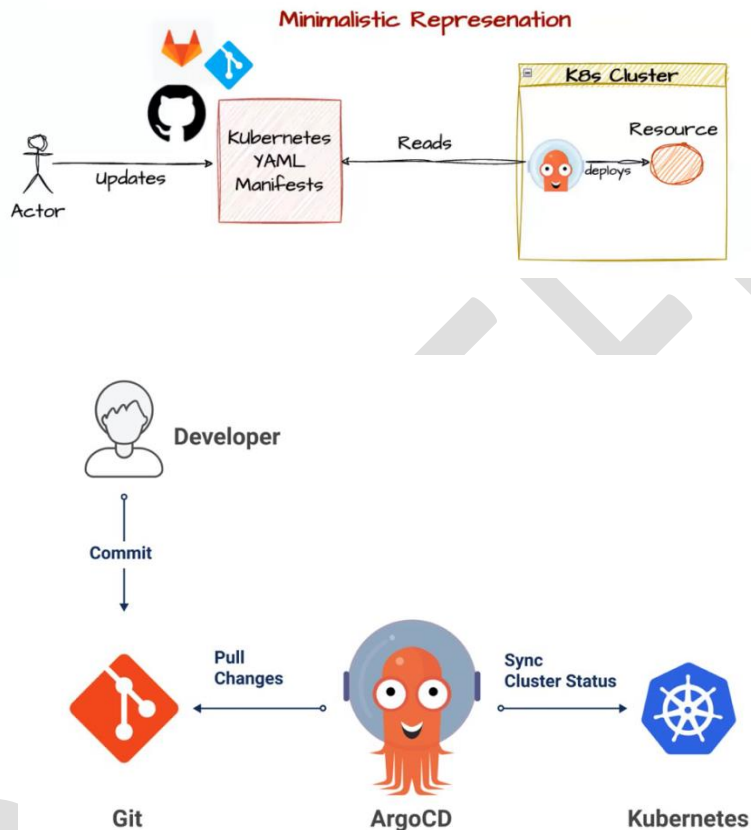


ARGOCD

WHAT IS GITOPS ?

GITOPS uses GIT as a single source of truth to deliver applications and infrastructure.



What is ArgoCD?

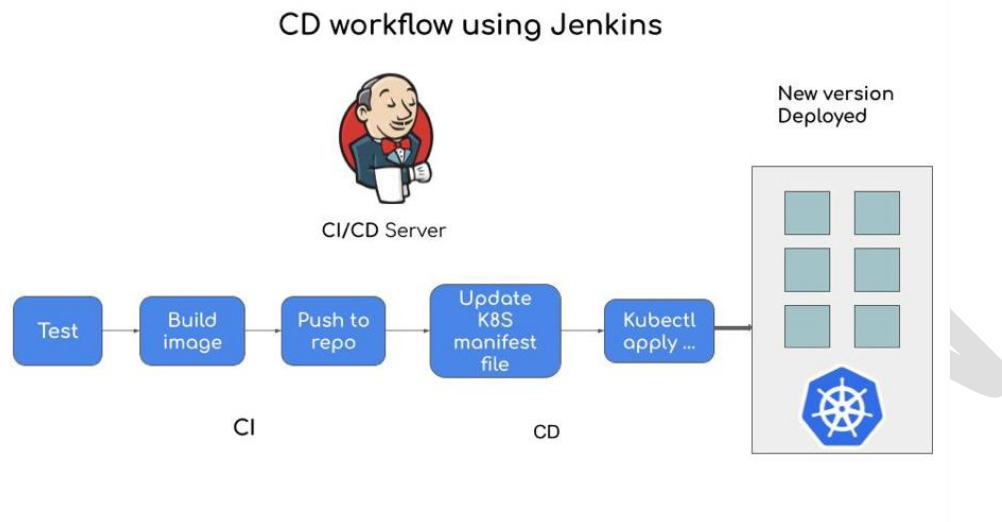
ArgoCD is a Kubernetes-native continuous deployment (CD) tool. Unlike external CD tools that only enable push-based deployments, Argo CD can pull updated code from Git repositories and deploy it directly to Kubernetes resources. It enables developers to manage both infrastructure configuration and application updates in one system.

There are many features of ArgoCD :

1. Whole Kubernetes configuration defined as code in GIT Repository.
2. Web user interface and command-line interface (CLI).
3. It has easy rollback mechanism through which we can rollback our changes to any previous state.
4. It has cluster disaster recovery feature which helps to create the exact cluster if cluster dies because all codes are present in GIT.
5. It contains Kubernetes access control with GIT.

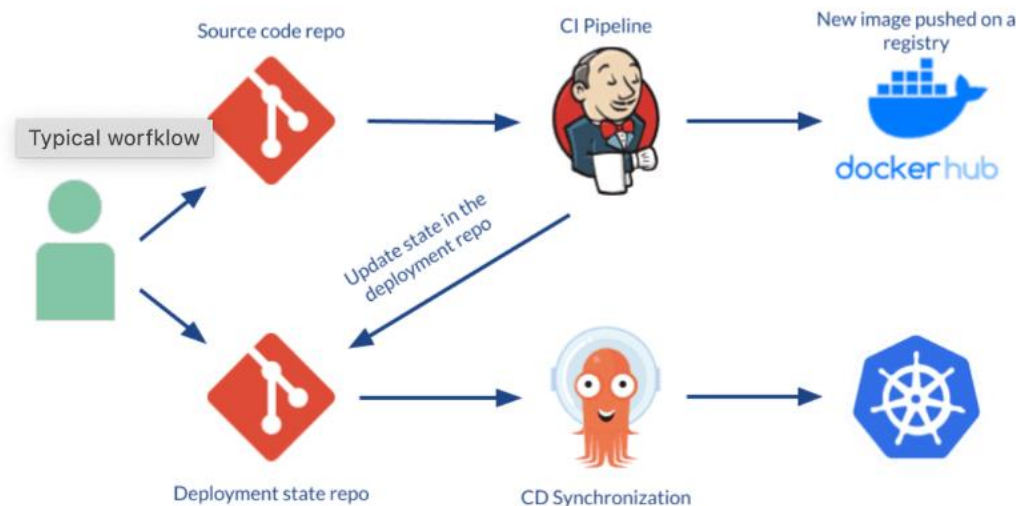
6. Single sign-on (SSO) with providers such as GitLab, GitHub, Microsoft, OAuth2, OIDC, LinkedIn, LDAP, and SAML 2.0
7. Support for webhooks triggering actions in GitLab, GitHub, and BitBucket.
8. Ability to visualize deployment issues, detect and remediate configuration drift.

What are the challenges which Jenkins have and ArgoCD solves ?



Challenges with this Approach ??

1. We need to install tools like kubectl on Jenkins.
2. Give access of Kubernetes cluster in Jenkins.
3. Configure access to cloud platform in Jenkins.
4. There is no visibility of deployment status in Jenkins.



CD Workflow using ArgoCD

1. A developer makes changes to an application, pushing a new version of Kubernetes resource definitions to a Git repo.
2. Continuous integration is triggered, resulting in a new container image saved to a registry.

3. A developer issues a pull request, changing Kubernetes manifests, which are created either manually or automatically.
4. The pull request is reviewed and changes are merged to the main branch. This triggers a webhook which tells Argo CD a change was made.
5. Argo CD clones the repo and compares the application state with the current state of the Kubernetes cluster. It applies the required changes to cluster configuration.
6. Kubernetes uses its controllers to reconcile the changes required to cluster resources, until it achieves the desired configuration.
7. Argo CD monitors progress and when the Kubernetes cluster is ready, reports that the application is in sync.
8. ArgoCD also works in the other direction, monitoring changes in the Kubernetes cluster and discarding them if they don't match the current configuration in Git.

STEP 1: SETUP KOPS OR MINIKUBE CLUSTER.

- In my case I set-up kops cluster

```

root@ip-172-31-39-172~
[root@ip-172-31-39-172 ~]# kubectl get no
NAME                                STATUS    ROLES    AGE     VERSION
i-079ebdc70236ac813               Ready    control-plane   8m42s   v1.30.2
i-0dba4fc6dacefb1c1               Ready    node          6m20s   v1.30.2
i-0fc6e27d9b568ca5e               Ready    node          6m20s   v1.30.2
root@ip-172-31-39-172 ~]#

```

STEP 2: INSTALL ARGOCD

`kubectl create namespace argocd`

`kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml`

STEP 3: EXPOSE ARGO CD SERVER

By default, Argo CD is not accessible outside the cluster. To expose it, you can use a LoadBalancer service (since your kOps cluster should be configured to support it).

1. Edit the Argo CD service to change the argocd-server service type:

`kubectl patch svc argocd-server -n argocd -p '{"spec": {"type": "LoadBalancer"}}'`

(or)

you can expose the Argo CD server externally by editing the Argo CD server service type.

`kubectl edit svc argocd-server -n argocd`

Change the type from ClusterIP to LoadBalancer to get an external IP address.

```

argocd-server-70b5899ed/-xvcfw      1/1      Running    0          54s
root@ip-172-31-39-172 ~]# kubectl get svc -n argocd
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)                                AGE
argocd-applicationset-controller    ClusterIP           100.71.228.158   <none>            7000/TCP, 8080/TCP                    96s
argocd-dex-server                   ClusterIP           100.67.52.93     <none>            5556/TCP, 5557/TCP, 5558/TCP          96s
argocd-metrics                      ClusterIP           100.68.117.1     <none>            8082/TCP                              96s
argocd-notifications-controller-metrics ClusterIP           100.64.78.36     <none>            9001/TCP                              96s
argocd-redis                        ClusterIP           100.68.51.168    <none>            6379/TCP                              96s
argocd-repo-server                  ClusterIP           100.65.208.113   <none>            8081/TCP, 8084/TCP                    96s
argocd-server                       ClusterIP           100.67.447.232   <none>            80/TCP, 443/TCP                      96s
argocd-server-metrics              ClusterIP           100.67.77.124    <none>            8005/TCP                              96s
root@ip-172-31-39-172 ~]# kubectl edit svc argocd-server -n argocd
service/argocd-server edited

```

```

app.kubernetes.io/name: argocd-server
sessionAffinity: None
type: LoadBalancer
status:
  loadBalancer:
    ingress:
      - hostname: adc00b6fdd2f7477cad3895bee94ab1f-676091007.us-east-2.elb.amazonaws.com
50,1      Bot

```

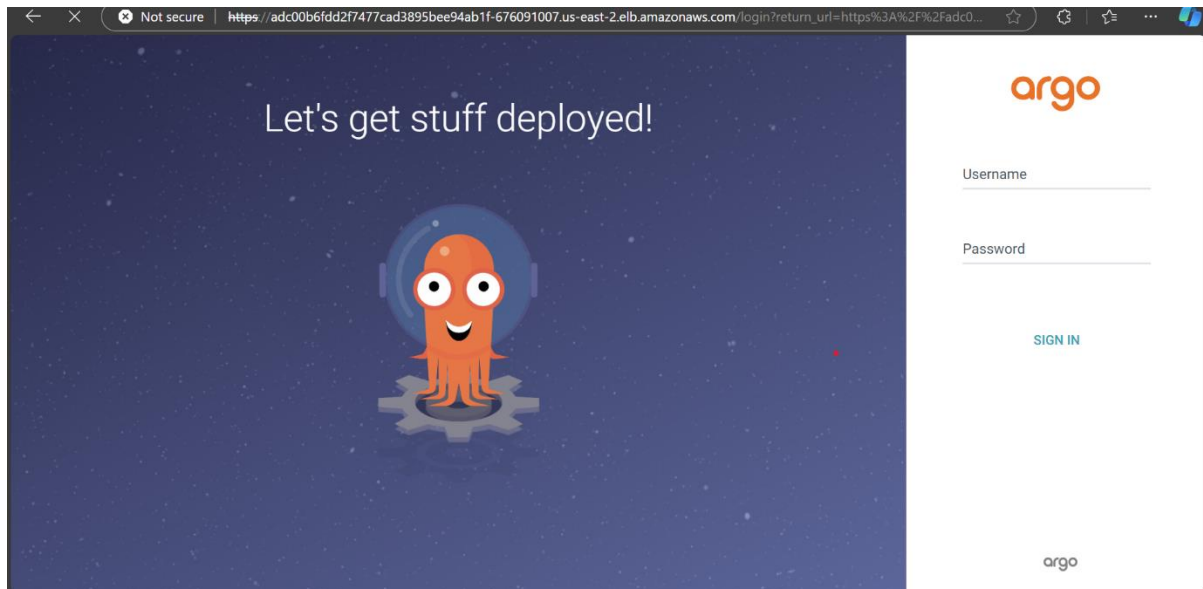
STEP 4: ACCESS ARGO CD

Once you have the LoadBalancer IP or DNS, you can access the Argo CD UI at <http://<LoadBalancer-I>.

```
root@ip-172-31-39-172 ~# kubectl get svc -n argocd
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
argocd-applicationset-controller	ClusterIP	100.71.228.158	<none>	7000/TCP,8080/TCP
argocd-dex-server	ClusterIP	100.67.32.93	<none>	5556/TCP,5557/TCP,5558/TCP
argocd-metrics	ClusterIP	100.68.117.1	<none>	8082/TCP
argocd-notifications-controller-metrics	ClusterIP	100.64.78.36	<none>	9001/TCP
argocd-redis	ClusterIP	100.68.51.168	<none>	6379/TCP
argocd-reso-server	ClusterIP	100.65.208.113	<none>	8081/TCP,8084/TCP
argocd-server	LoadBalancer	100.67.247.232	adc00b6fdd2f7477cad3895bee94ab1f-676091007.us-east-2.elb.amazonaws.com	80:30195/TCP,443:31164/TCP
argocd-server-metrics	ClusterIP	100.67.7.134	<none>	8083/TCP

```
root@ip-172-31-39-172 ~#
```



STEP 5: RETRIEVE THE INITIAL ADMIN PASSWORD

To log in, use the default admin credentials. Get the password with:

```
kubectl get secret argocd-initial-admin-secret -n argocd -o jsonpath="{.data.password}" | base64 -d
```

Use admin as the username and the retrieved password.

(or)

Username: admin

To get password:

Get secrets :

```
kubectl get secrets -n argocd
```

```
root@ip-172-31-39-172 ~# kubectl get secret -n argocd
```

NAME	TYPE	DATA	AGE
argocd-initial-admin-secret	Opaque	1	13m
argocd-notifications-secret	Opaque	0	13m
argocd-redis	Opaque	1	13m
argocd-secrets	Opaque	1	13m

```
root@ip-172-31-39-172 ~# kubectl edit secret argocd-initial-admin-secret -n argocd
```

```
Edit cancelled, no changes made.
root@ip-172-31-39-172 ~# echo SGRsOWRPuzVQakpOSlpYTA==
SGRsOWRPuzVQakpOSlpYTA==
root@ip-172-31-39-172 ~# echo SGRsOWRPuzVQakpOSlpYTA== |base64 --decode
-bash: base64: command not found
root@ip-172-31-39-172 ~# echo SGRsOWRPuzVQakpOSlpYTA== |base64 --decode
d19d05SP1jN3Xl1root@ip-172-31-39-172 ~#
```

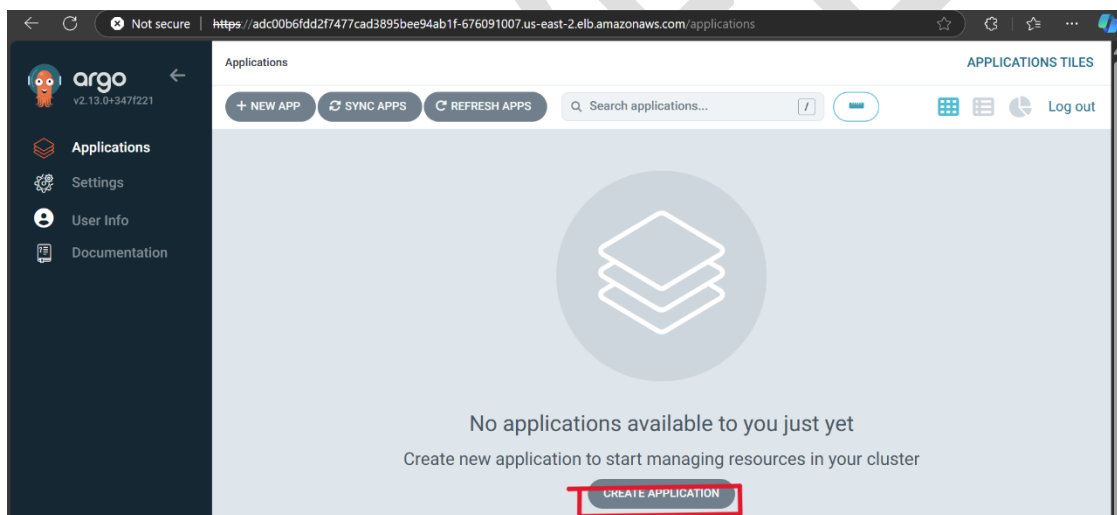
- Copy the password and decode the it.

```
# Please edit the object below. Lines beginning with a '#' will be ignored,
# and an empty file will abort the edit. If an error occurs while saving this file will be
# reopened with the relevant failures.
#
apiVersion: v1
data:
  password: SGRsOWRPuzVQakpOSlpYTA==
kind: Secret
metadata:
  creationTimestamp: "2024-11-06T04:20:40Z"
  name: argocd-initial-admin-secret
  namespace: argocd
  resourceVersion: "3089"
  uid: b35095c6-56ff-4d49-89e3-3e0ac39596ec
type: Opaque
```

```
root@ip-172-31-39-172:~# kubectl get secret -n argocd
NAME                                TYPE      DATA      AGE
argocd-initial-admin-secret         Opaque    1           13m
argocd-notifications-secret         Opaque    0           13m
argocd-redis                        Opaque    1           13m
argocd-secret                       Opaque    1           13m
root@ip-172-31-39-172:~# kubectl edit secret argocd-initial-admin-secret -n argocd
edit cancelled, no changes made.
root@ip-172-31-39-172:~# echo SGRsOWRPuzVQakpOSlpYTA==
SGRsOWRPuzVQakpOSlpYTA==
root@ip-172-31-39-172:~# echo SGRsOWRPuzVQakpOSlpYTA== |base64 --decode
-bash: base64: command not found
root@ip-172-31-39-172:~# echo SGRsOWRPuzVQakpOSlpYTA== |base64 --decode
d19d055p1jN1Zxl1root@ip-172-31-39-172:~#
```

STEP 6: CONNECT A GIT REPOSITORY

1. Go to the Argo CD UI and log in.
2. Set up a new project with your Git repository details.
3. Configure the repository settings, sync policies, and deployment specifications.

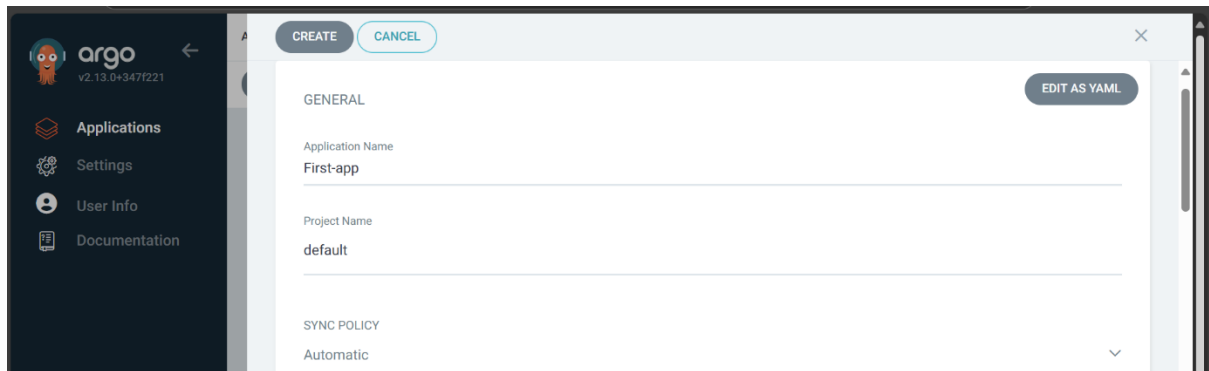


- Click on Create application.
- **Application Name:** Choose a unique name for the application.
- **Project:** Select a project (default project is available by default).
- **Sync Policy:** This defines whether Argo CD should automatically sync the application to match the Git repository or wait for manual approval.
 - *Automatic:* Argo CD will automatically apply changes detected in the Git repo.
 - *Manual:* You need to trigger deployments manually from the UI or CLI.
- **Repository URL:** URL of your Git repository containing the manifests.
- **Revision:** Specify a branch (e.g., main), tag, or commit SHA for the desired state version.
- **Path:** Path within the Git repo where Kubernetes manifests or Helm charts are located.

2. Specify the Target Cluster and Namespace

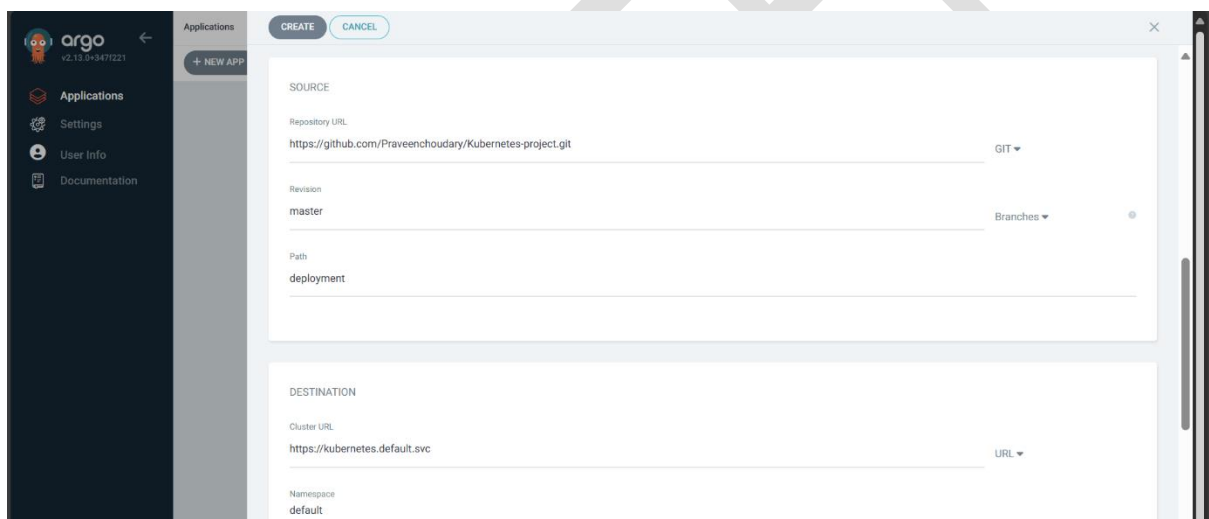
Define where you want the application to be deployed:

- **Cluster URL:** Usually, this is set to the same cluster where Argo CD is running (<https://kubernetes.default.svc>), but it can be a different cluster if you've configured multi-cluster support.
- **Namespace:** The Kubernetes namespace in which to deploy the application. Make sure the namespace exists in the cluster or create it manually.



The screenshot shows the 'CREATE' dialog in the Argo CD web interface. The left sidebar contains the Argo logo, version 'v2.13.0+347f221', and navigation links for Applications, Settings, User Info, and Documentation. The main panel is titled 'GENERAL' and has an 'EDIT AS YAML' button. It contains the following fields:

- Application Name:** First-app
- Project Name:** default
- SYNC POLICY:** Automatic

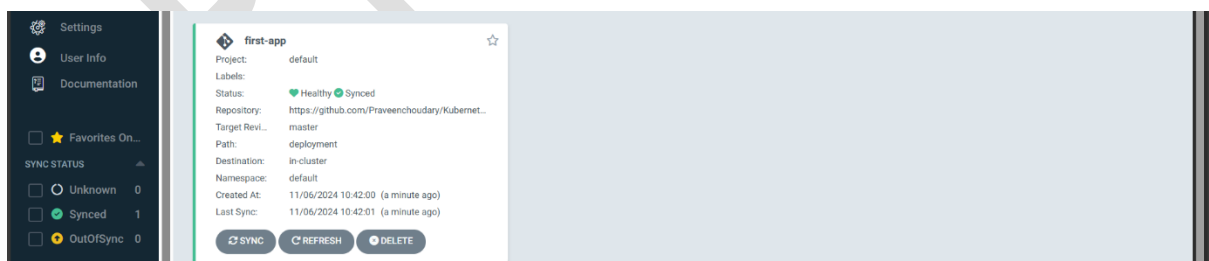


The screenshot shows the 'CREATE' dialog with the 'SOURCE' and 'DESTINATION' tabs. The 'SOURCE' tab contains the following fields:

- Repository URL:** <https://github.com/Praveenchoudary/Kubernetes-project.git>
- Revision:** master
- Path:** deployment

The 'DESTINATION' tab contains the following fields:

- Cluster URL:** <https://kubernetes.default.svc>
- Namespace:** default

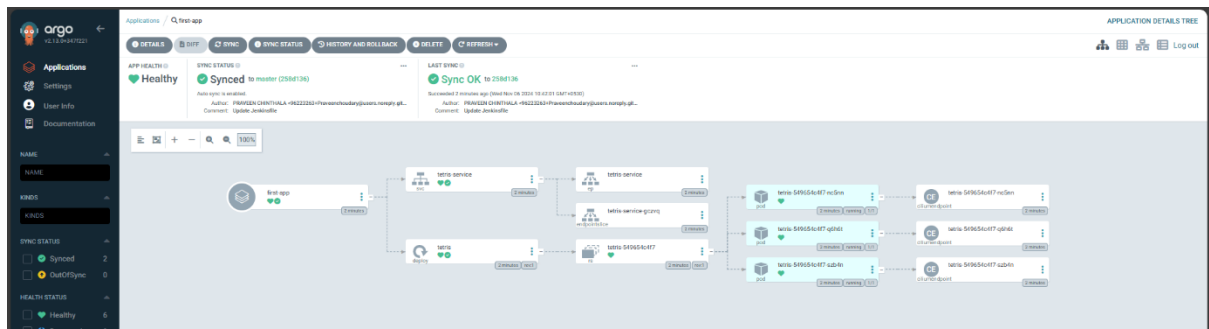


The screenshot shows the 'first-app' details view in the Argo CD web interface. The left sidebar contains the 'Settings' link and a 'SYNC STATUS' section with a legend: Unknown (0), Synced (1), and OutOfSync (0). The main panel displays the following details for 'first-app':

- Project:** default
- Labels:**
- Status:** Healthy Synced
- Repository:** <https://github.com/Praveenchoudary/Kubernetes-project.git>
- Target Revision:** master
- Path:** deployment
- Destination:** in-cluster
- Namespace:** default
- Created At:** 11/06/2024 10:42:00 (a minute ago)
- Last Sync:** 11/06/2024 10:42:01 (a minute ago)

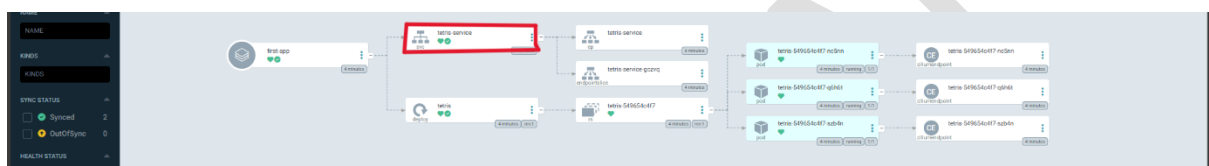
At the bottom, there are three buttons: SYNC, REFRESH, and DELETE.

- The app is synced successfully.

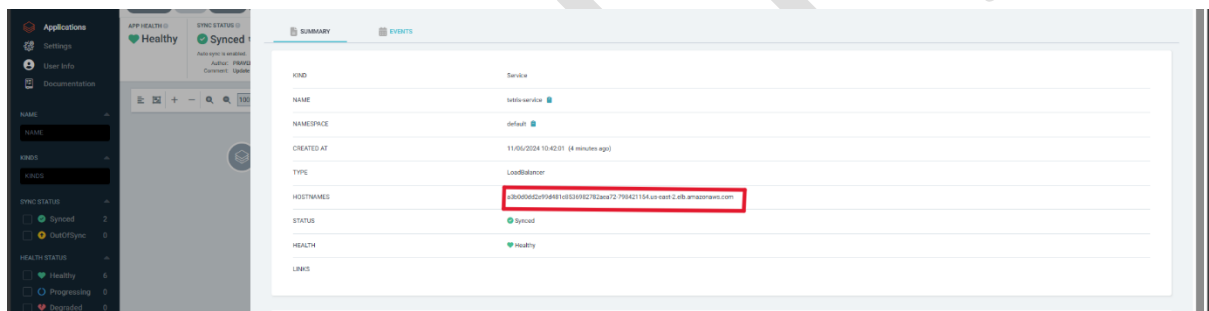


```
[root@ip-172-31-39-172 ~]# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
tetris-549654c4f7-nc5nn            1/1     Running   0           2m55s
tetris-549654c4f7-q6h6t            1/1     Running   0           2m55s
tetris-549654c4f7-szb4n            1/1     Running   0           2m55s
[root@ip-172-31-39-172 ~]#
```

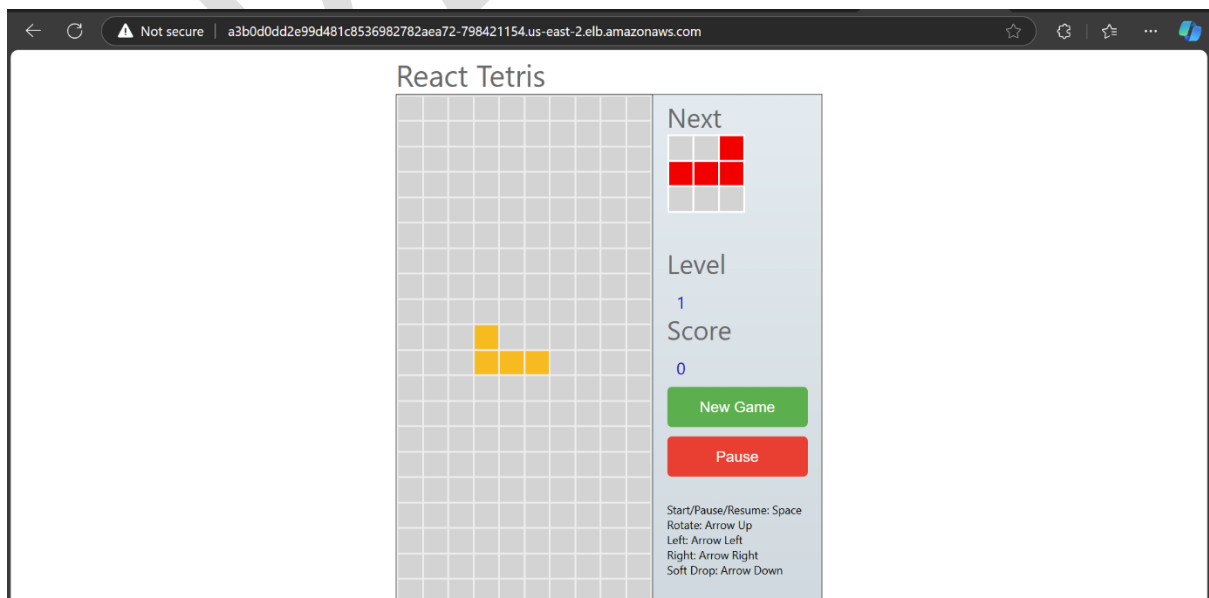
- Even when You see the k8s cluster we get running pods.



- Click on service



- Copy the LoadBalancer DNS



- Access the application using LoadBalancer DNS.

If you make manual changes to resources in your Kubernetes cluster, Argo CD will notice these changes and mark the application as "Out of Sync" with what's in the Git repository.

Here's how to handle it:

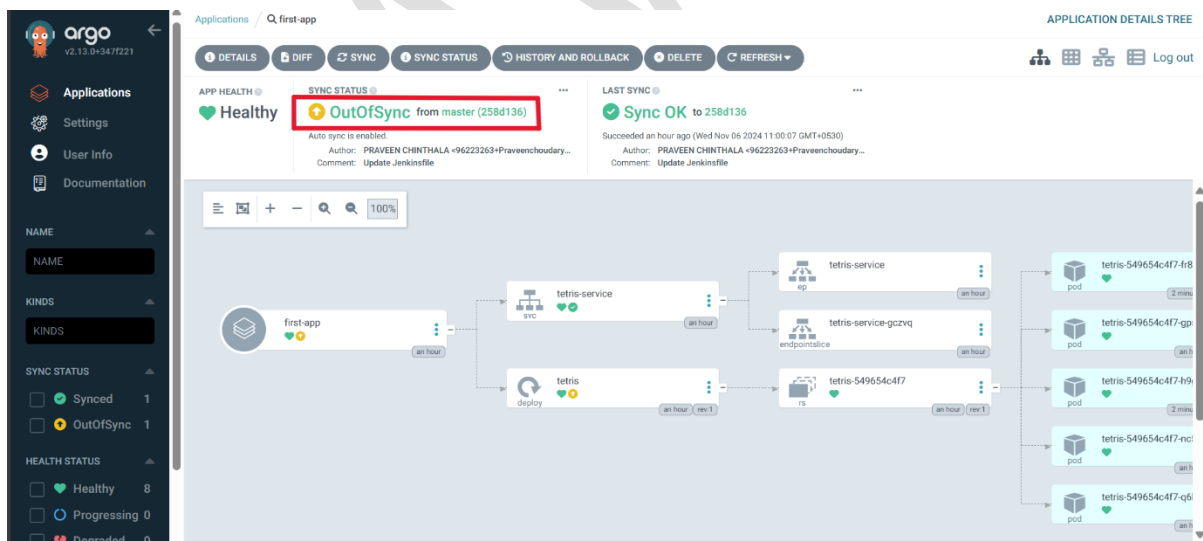
1. **Automatic Sync:** If auto-sync is enabled, Argo CD will automatically revert the changes in the cluster to match the Git state.
2. **Manual Sync:** If using manual sync, you'll see an "Out of Sync" status in the Argo CD UI. You can click **Sync** in the UI or run `argocd app sync <app-name>` in the CLI to manually reapply the Git configuration to the cluster.
3. **Keeping Manual Changes:** If you want to keep the manual changes, update the same changes in your Git repository, then commit and push. Argo CD will detect this update and sync accordingly.

```
namespace: default
resourceVersion: "18216"
uid: 2f03141a-d95f-4ece-9c01-73733c25678f
spec:
  progressDeadlineSeconds: 600
  replicas: 3
  revisionHistoryLimit: 10
  selector:
```

- Now, let's manually change the deployment yaml file from the cluster configuration → Changed replica count from 3 to 5.

```
namespace: default
resourceVersion: "18216"
uid: 2f03141a-d95f-4ece-9c01-73733c25678f
spec:
  progressDeadlineSeconds: 600
  replicas: 5
  revisionHistoryLimit: 10
  selector:
```

It will discard the changes which we have done manually because ArgoCD follows Git as the single source of truth. So, what happens at the background is it checks the cluster desired with the actual state of Git and revert the changes.



When you make changes directly in your Kubernetes cluster (e.g., updating a deployment or modifying resources without using Argo CD), Argo CD detects that the cluster's actual state no longer matches the desired state defined in the Git repository. This state mismatch causes Argo CD to label the application as "Out of Sync."

```
SYNC STATUS
[ ] Synced 1
[ ] OutOfSync 1

106 uid: 2f03141a-d95f-4ece-9c01-73733c25678f
107 spec:
108   progressDeadlineSeconds: 600
109   replicas: 5
110   revisionHistoryLimit: 10
111   selector:

106 uid: 2f03141a-d95f-4ece-9c01-73733c25678f
107 spec:
108   progressDeadlineSeconds: 600
109   replicas: 3
110   revisionHistoryLimit: 10
111   selector:
```